

FINAL PROJECT REPORT

Customer Retail Data Analysis Using Machine Learning Models

Prepared by: N. Abishan

1. Introduction

Machine Learning (ML) is a fundamental branch of Artificial Intelligence that empowers computational systems to derive insights from data and generate informed predictions without being explicitly programmed. This report documents the development, implementation, and evaluation of multiple Machine Learning classification algorithms applied to a customer retail dataset. The project encompasses the complete ML pipeline, including data preprocessing, model training, performance evaluation, and comparative analysis of algorithmic approaches.

2. Project Objectives

The primary aim of this project is to implement and systematically compare multiple Machine Learning classification algorithms applied to a customer retail dataset. The specific objectives pursued throughout this study are as follows:

- Data preprocessing and cleaning
- Identification and treatment of missing values
- Encoding of categorical variables into numerical representations
- Exploratory data visualisation
- Model training and predictive inference
- Quantitative model evaluation
- Comparative performance analysis across algorithms

3. Dataset Description

The dataset employed in this project is a customer retail dataset comprising sales transaction records. The following attributes were utilised as input features for model training:

- Quantity — the number of units sold per transaction
- UnitPrice — the price per individual unit
- Country — the geographic origin of the transaction

Additional columns present in the dataset, used for contextual reference, include: InvoiceNo, StockCode, Description, InvoiceDate, and CustomerID.



Figure 1: Dataset Overview Screenshot

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	01-12-2010 08:26	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	01-12-2010 08:26	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	01-12-2010 08:26	2.75	17850.0	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	01-12-2010 08:26	3.39	17850.0	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	01-12-2010 08:26	3.39	17850.0	United Kingdom

Figure 2: Dataset Columns and Structure

4. Tools and Technologies

The following software tools and libraries were utilised throughout the project:

- Python — primary programming language
- Google Colab — cloud-based development environment
- Pandas — data manipulation and analysis
- NumPy — numerical computation
- Matplotlib — data visualisation and graphing
- Scikit-learn — machine learning model implementation and evaluation

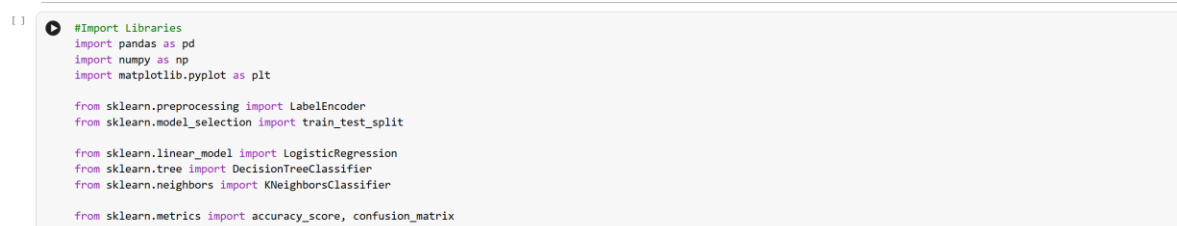


Figure 3: Tools and Library Imports

5. Data Preprocessing

Data preprocessing constitutes a critical preliminary stage in any Machine Learning workflow, ensuring that the input data is clean, consistent, and suitable for model consumption. The following preprocessing steps were systematically applied:

5.1 Handling Missing Values

Instances containing missing or null values were identified and subsequently removed from the dataset using the `dropna()` function, thereby ensuring data integrity across all feature dimensions.

```
[ ]  
# Handle Missing Values  
df.isnull().sum()  
  
# Remove missing values  
df = df.dropna()
```

Figure 4: Missing Value Removal using `dropna()`

5.2 Encoding Categorical Data

The Country attribute contained non-numeric string values. To enable compatibility with numerical ML algorithms, the `LabelEncoder` utility from Scikit-learn was employed to transform these textual labels into corresponding integer representations.

```
[ ]  
# Encode Country Column  
encoder = LabelEncoder()  
  
df['Country'] = encoder.fit_transform(df['Country'])  
  
df.head()
```

```
▼ ...  
   Quantity  UnitPrice  Country  
0         6         2.55        36  
1         6         3.39        36  
2         8         2.75        36  
3         6         3.39        36  
4         6         3.39        36
```

Figure 5: Label Encoding of Categorical Variables

5.3 Feature Selection

Three features were selected as model inputs based on their relevance to the classification task: Quantity, UnitPrice, and Country.

5.4 Target Variable Construction

A binary target variable was engineered from the Quantity feature using the following threshold logic:

- Quantity greater than 10 was assigned a class label of 1 (high volume)
- Quantity of 10 or fewer was assigned a class label of 0 (standard volume)

```
[ ] #Create Target Column
df['Target'] = np.where(df['Quantity'] > 10, 1, 0)
df.head()
```

	Quantity	UnitPrice	Country	Target
0	6	2.55	36	0
1	6	3.39	36	0
2	8	2.75	36	0
3	6	3.39	36	0
4	6	3.39	36	0

Figure 6: Target Variable Creation Code

6. Data Visualisation

Visualisation techniques were employed to facilitate a deeper understanding of the dataset's distribution and the comparative performance of the trained models. The following charts were produced using the Matplotlib library:

- Customer Distribution Graph — illustrating the spread of customer transactions across geographic regions
- Model Accuracy Comparison Graph — providing a side-by-side comparison of predictive accuracy across all models



Figure 7: Customer Distribution Graph

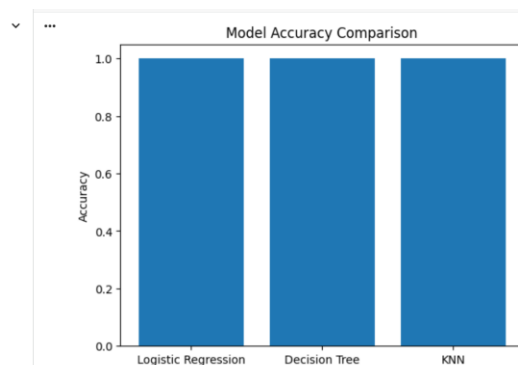


Figure 8: Model Accuracy Comparison Graph

7. Machine Learning Models

Three supervised classification algorithms were selected and implemented for this analysis. Each model represents a distinct learning paradigm, facilitating a comprehensive comparative evaluation.

7.1 Logistic Regression

Logistic Regression is a well-established supervised learning algorithm used for binary classification tasks. It models the probability of class membership by applying a logistic (sigmoid) function to a linear combination of input features, making it particularly effective for linearly separable datasets.

7.2 Decision Tree Classifier

The Decision Tree algorithm is a non-parametric, tree-structured model that partitions the feature space through a sequence of hierarchical decision rules. It is capable of capturing non-linear relationships and is valued for its interpretability and visual transparency.

7.3 K-Nearest Neighbors (KNN)

The K-Nearest Neighbors algorithm is an instance-based, non-parametric classifier that assigns a class label to a data point based on the majority class among its K closest neighbours in the feature space. Its simplicity and adaptability make it a widely applied baseline classifier.

8. Model Training and Testing

The dataset was partitioned into training and testing subsets in an 80:20 ratio, ensuring that the majority of the data was used for model learning whilst a representative holdout set was retained for unbiased performance evaluation. This split was performed using the `train_test_split()` function from the Scikit-learn library.

9. Evaluation Metrics

Two standard evaluation metrics were employed to assess the classification performance of each model:

9.1 Accuracy Score

The Accuracy Score quantifies the proportion of correctly classified instances out of the total number of predictions, providing a high-level measure of overall model performance.

9.2 Confusion Matrix

The Confusion Matrix provides a detailed breakdown of prediction outcomes, enabling deeper insight into the distribution of true positives, false positives, true negatives, and false negatives for each model.

10. Results and Discussion

Upon completion of training, the predictive accuracy of all three models was recorded and compared. The results are summarised in the table below:

Machine Learning Model	Accuracy Score
Logistic Regression	1.0000 (100%)
Decision Tree Classifier	1.0000 (100%)
K-Nearest Neighbors (KNN)	0.9999 (99.99%)

All three models delivered exceptional classification accuracy on this dataset. Both Logistic Regression and the Decision Tree Classifier achieved perfect accuracy scores of 100%, whilst the KNN algorithm attained a near-perfect score of 99.99%.

```
[ ] #Logistic Regression
log_model = LogisticRegression()

log_model.fit(X_train, y_train)

log_pred = log_model.predict(X_test)

log_acc = accuracy_score(y_test, log_pred)

print("Logistic Regression Accuracy:", log_acc)

print(confusion_matrix(y_test, log_pred))

▼ ... Logistic Regression Accuracy: 1.0
[[82035  0]
 [  0 26347]]
```

Figure 9: Logistic Regression — Accuracy and Confusion Matrix

```
[ ] #Decision Tree
dt_model = DecisionTreeClassifier()

dt_model.fit(X_train, y_train)

dt_pred = dt_model.predict(X_test)

dt_acc = accuracy_score(y_test, dt_pred)

print("Decision Tree Accuracy:", dt_acc)

print(confusion_matrix(y_test, dt_pred))

▼ Decision Tree Accuracy: 1.0
[[82035  0]
 [  0 26347]]
```

Figure 10: Decision Tree Classifier — Accuracy and Confusion Matrix

```
[ ] #KNN Model
knn_model = KNeighborsClassifier(n_neighbors=5)

knn_model.fit(X_train, y_train)

knn_pred = knn_model.predict(X_test)

knn_acc = accuracy_score(y_test, knn_pred)

print("KNN Accuracy:", knn_acc)

print(confusion_matrix(y_test, knn_pred))

KNN Accuracy: 0.9999907733756528
[[ 82835   0]
 [   1 26346]]
```

Figure 11: K-Nearest Neighbors — Accuracy and Confusion Matrix

11. Conclusion

This project successfully demonstrated the end-to-end implementation of multiple supervised Machine Learning classification algorithms applied to a customer retail dataset. All stages of the standard ML pipeline were executed, including data preprocessing, feature engineering, model training, prediction, and quantitative evaluation.

The project yielded the following key learning outcomes:

- A practical understanding of supervised learning methodologies
- Hands-on experience with classification algorithm implementation
- Proficiency in data preprocessing techniques
- Application of model evaluation metrics
- Comparative performance analysis across algorithmic paradigms

The variation in accuracy scores observed across the three models underscores the importance of algorithm selection in Machine Learning workflows. While all three classifiers performed exceptionally on this dataset, marginal differences in accuracy highlight the influence of model complexity and underlying assumptions on predictive outcomes.

12. References

- Scikit-learn: Machine Learning in Python — <https://scikit-learn.org/stable/documentation.html>
- Google Colab — <https://colab.research.google.com>
- Python Programming Language — <https://docs.python.org/3/>
- Pandas Documentation — <https://pandas.pydata.org/docs/>